

ml sample report.docx

 Shri Vile Parle Kelavani Mandal

Document Details

Submission ID

trn:oid:::9832:134493088

Submission Date

Apr 7, 2026, 10:28 PM GMT+5:30

Download Date

Apr 7, 2026, 10:36 PM GMT+5:30

File Name

ml sample report.docx

File Size

2.5 MB

51 Pages

7,250 Words

43,457 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



1. Abstract

Every day, thousands of vehicles move through highways, creating patterns that often go unnoticed until congestion becomes a problem. This project, “*TrafficIQ — Metro Interstate Traffic Volume Prediction*”, began with a simple question: *Can we predict traffic before it actually builds up?*

Starting with raw traffic and weather data collected over several years, the journey involved cleaning inconsistent data, handling missing values, and uncovering hidden patterns within time-based behavior. One of the key challenges was dealing with irregular time gaps in the dataset, which required a more thoughtful, segment-based approach to ensure accurate feature creation.

As the project progressed, it became clear that understanding traffic is not just about current conditions, but also about learning from the immediate past. By introducing lag and rolling features, the model was able to capture how recent traffic influences future flow. Multiple machine learning models were explored, compared, and refined, ultimately leading to a highly accurate Tuned XGBoost model achieving an R^2 score of 0.9853.

Beyond prediction, this project focuses on explainability. Using SHAP analysis, the system not only predicts traffic volume but also explains *why* a certain prediction is made, making the model more transparent and trustworthy.

To make the solution practical and interactive, a Streamlit-based dashboard was developed, allowing users to explore data, compare models, and generate real-time predictions.

In essence, this project demonstrates how data-driven insights can transform traffic management from a reactive process into a proactive system, enabling smarter decisions and more efficient transportation planning.

2. Introduction

Every day, thousands of vehicles move through highways, creating patterns that often go unnoticed until congestion becomes a problem. This project, “*TrafficIQ — Metro Interstate Traffic Volume Prediction*”, began with a simple question: *Can we predict traffic before it actually builds up?*

Starting with raw traffic and weather data collected over several years, the journey involved cleaning inconsistent data, handling missing values, and uncovering hidden patterns within time-based behavior. One of the key challenges was dealing with irregular time gaps in the dataset, which required a more thoughtful, segment-based approach to ensure accurate feature creation.

As the project progressed, it became clear that understanding traffic is not just about current conditions, but also about learning from the immediate past. By introducing lag and rolling features, the model was able to capture how recent traffic influences future flow. Multiple machine learning models were explored, compared, and refined, ultimately leading to a highly accurate Tuned XGBoost model achieving an R^2 score of 0.9853.

Beyond prediction, this project focuses on explainability. Using SHAP analysis, the system not only predicts traffic volume but also explains *why* a certain prediction is made, making the model more transparent and trustworthy.

To make the solution practical and interactive, a Streamlit-based dashboard was developed, allowing users to explore data, compare models, and generate real-time predictions.

In essence, this project demonstrates how data-driven insights can transform traffic management from a reactive process into a proactive system, enabling smarter decisions and more efficient transportation planning.

2.1 Problem Statement

Traffic congestion remains a persistent issue in metropolitan areas, leading to increased travel time, fuel consumption, and environmental impact. Most existing systems rely on real-time monitoring, which limits their ability to prevent congestion before it occurs.

The problem addressed in this project is:

“To develop a machine learning-based system capable of accurately predicting hourly traffic volume using historical traffic data, time-based features, and weather conditions.”

The system must:

- Handle real-world data challenges such as missing values and time gaps
- Capture temporal dependencies in traffic patterns
- Provide high prediction accuracy
- Offer explainability for model decisions

By solving this problem, the project aims to enable **proactive traffic management** rather than reactive response.

2.2 Objective of the Project

The primary objective of this project is to design and implement a robust machine learning system for predicting metro interstate traffic volume on an hourly basis.

The specific objectives are as follows:

- To analyze and understand traffic patterns using historical data
- To perform effective data preprocessing, including handling missing values and time gaps
- To engineer meaningful features that capture temporal and behavioral patterns in traffic
- To build and compare multiple machine learning models for prediction

- To optimize model performance using hyperparameter tuning techniques
- To evaluate model performance using appropriate metrics such as R^2 , RMSE, and MAE
- To incorporate explainability techniques (SHAP) to interpret model predictions
- To develop an interactive dashboard for visualization and real-time prediction

2.3 Scope and Applications

The scope of this project extends beyond academic implementation and has practical relevance in real-world traffic management systems. By accurately predicting traffic volume, the system can support various applications:

- ***Traffic Signal Optimization:*** Adjust signal timings based on predicted traffic flow to reduce congestion.
- ***Urban Planning:*** Assist city planners in designing better road infrastructure using long-term traffic patterns.
- ***Navigation Systems:*** Improve route recommendations and estimated travel times in navigation applications.
- ***Emergency Response Management:*** Enable faster route planning for emergency vehicles by anticipating traffic conditions.
- ***Environmental Monitoring:*** Estimate areas of high congestion and associated emissions for better environmental policies.
- ***Smart City Systems:*** Integrate with intelligent transportation systems to enable data-driven decision-making.

The project is currently limited to historical data analysis and does not incorporate real-time streaming data. However, the methodology and system design can be extended to real-time deployment in future implementations.

3. Literature Review

3. Literature Review

Traffic prediction has been an active research area within intelligent transportation systems for several decades. With the rise of urbanization and smart city initiatives, researchers have increasingly focused on developing accurate and efficient models to forecast traffic flow using historical and real-time data.

This section reviews previous work in two key areas:

- Traffic prediction techniques
- Time series machine learning approaches

It also highlights key limitations in existing research that motivate this project.

3.1 Previous Work in Traffic Prediction

Initial research into traffic prediction utilised a number of statistical time series models including ARIMA and SARIMA. These models were widely used and well established due to their simplicity and strong theoretical basis. However, they were found to be inadequate in capturing the many complex dynamics that occur within traffic flow.

While the aforementioned models have achieved considerable success in representing vehicular flow from both empirical and analytical perspectives, they rely upon crucial assumptions that include linear relationships among dynamic traffic parameters. In reality, however, traffic evolution is influenced by a host of complex, highly nonlinear factors.

Introducing intelligence to traffic flow prediction tasks, researchers have started adopting machine learning techniques like regression, decision trees and ensemble learning methods that can effectively handle high-dimensional, non-linear traffic data.

Besides sequence-to-sequence models, in recent years deep learning models gained significant attention.

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM) networks

- Graph-based neural networks

The recent models are able to learn spatio-temporal features which are critical in traffic systems. For instance, the work of spatio-temporal graph convolutional networks present that the traffic flow not only has temporal dynamics but also the dynamics of relationships between the segments of road networks.

In addition to camera vision, modern traffic prediction systems can also utilize multiple sources of data such as weather, traffic sensors and GPS streams. Machine learning-based traffic prediction systems have proven themselves in practice and support the intelligent management of traffic congestion.

3.2 Previous Work in Time Series Machine Learning

Traffic prediction is treated as a time series forecasting problem: past observations of a time series (e.g. traffic speed) are used to predict future values of that time series.

Traditional time series techniques include:

- ARIMA (AutoRegressive Integrated Moving Average)
- Exponential smoothing methods

However current approaches to traffic prediction are mainly geared towards dealing with static patterns and are not effective at handling the often dynamic and irregular nature of real world traffic behaviour.

Machine learning approaches improved upon this by introducing:

- Feature-based learning (e.g., lag features, rolling statistics)
- Nonlinear modeling capabilities

Following from previous advances in time series forecasting, the last few years have seen deep learning outperform traditional models by effectively capturing long-term time series patterns. While the Long Short-Term Memory (LSTM) Recurrent Neural Network (RNN) and Gate Recurrent Unit (GRU) RNN have proven particularly effective at forecasting traffic

patterns, this research seeks to apply these models to an even more complex problem by utilizing the large number of cameras and higher spatial resolution available in the NYC Metro Area.

Existing work on temporal models has focused on capturing temporal dynamics of time series. However, it has become increasingly clear that periodicity, as well as external effects like weather or events, is crucial for understanding real-world data.

In addition, hybrid models have been recently proposed which combine statistical and deep learning approaches such as LSTM + ARIMA to further improve the predictive power.

3.3 Limitations in Existing Research

Despite significant advancements, several limitations remain in current traffic prediction systems:

1. Lack of Explainability

Almost all modern models, especially the deep learning models, are black-box models where it is not straightforward to interpret how individual predictions are made. Balancing accuracy and interpretability is a challenging task especially for traffic prediction models. Lack of transparency may prevent models from getting the trust and usability that they need in practice

2. Poor Feature Engineering

While some studies rely heavily on raw data without incorporating meaningful engineered features, traffic behavior is greatly affected by

- Time of day (hour, weekday)
- Recent traffic trends (lag features)
- Interactions between variables

Using poor feature engineering could really reduce the effectiveness of your models and prevent you from really seeing what's going on.

3. Inability to Capture Complex Dependencies

Traffic systems entail:

- Temporal dependencies (past traffic patterns)
- Spatial dependencies (interaction between road segments)

Most traditional models achieve either high accuracy or long lifetime, but not both and hence their prediction accuracy drops.

4. Over-Focus on Accuracy

There is much work attempting to improve accuracy, but little work exploring the possibility that other objectives, such as maximum reliability or minimum error, could be more desirable.

- Real-time applicability
- Model interpretability
- Practical deployment challenges

However, there remains a significant disconnection between current research models and clinical practice.

5. Data-Related Challenges

Existing studies also face issues such as:

- Handling missing or inconsistent data
- Managing large-scale datasets
- Dealing with irregular time intervals

Existing problems further complicate accurate prediction.

These problems have to be addressed to make prediction models reliable.

3.4 Research Gap

Although progress has been made in various studies addressing these requirements, there still remains a need for a comprehensive system for monitoring, prediction, decision support, data mining, and remote access for hydrological, meteorological, and oceanographic monitoring stations and weather radars.

- Combines high prediction accuracy with interpretability
- Clever feature engineering to realistically model traffic behavior.
- Takes care of all time-series data difficulties including missing data and inconsistencies.

- Practical solution over ideal, as opposed to only theoretical performance.

3.5 Motivation for the Proposed Work

This project addresses the identified gaps by:

- Use of more advanced feature engineering, primarily focusing on use of lag and rolling features.
- High accuracy achieved by utilising ensemble machine learning models.
- Applying SHAP explainability to interpret predictions
- Building an interactive dashboard for real-world usability

To bridge the gap between the growing amount of theoretical studies on traffic forecasting and the empirical needs for practicing traffic prediction, we propose a real-time traffic monitoring and forecasting system in this paper.

4.0 Dataset Description

The data I used for this data visualization project can be found online at the [UCI Machine Learning Repository](#) in the Metro Interstate Traffic Volume dataset. This data set contains several years worth of hourly traffic volume statistics recorded on multiple interstates in one metropolitan area.

Overview of the Dataset -

Attribute	Details
Total Records	48,204
Time Period	October 2012 – September 2018
Frequency	Hourly
Target Variable	Traffic Volume (vehicles/hour)

This dataset contains several types of data (weather, time and traffic count). Hence it is appropriate for time-series prediction problems.

Features in the Dataset

The dataset includes the following key attributes:

Feature Name	Type	Description
date_time	DateTime	Timestamp of the observation
traffic_volume	Integer	Number of vehicles per hour (Target)
temp	Float	Temperature (in Kelvin)
rain_1h	Float	Rainfall in the last hour (mm)
snow_1h	Float	Snowfall in the last hour (mm)
clouds_all	Integer	Cloud coverage (%)
weather_main	Categorical	General weather condition
weather_description	Categorical	Detailed weather description
holiday	Categorical	Holiday indicator

Data Challenges Identified

Even though this dataset has a lot of data, it has a number of real-world problems.

1. Missing Values:

The holiday column contained over 99% missing values and would have stopped processing if we went with it.

2. Duplicate Records:

After importing the data, I removed the few duplicated timestamps.

3. Time Gaps:

There were some gaps in the data – the biggest was over 300 days but there were some other large gaps as well. Some of these gaps were over several months, which meant that time features like days since last post and time since follower would not increase steadily over time.

4. Data Format Issues:

The timestamp column was provided as a string and was required to be converted to datetime format.

Importance of Dataset Understanding

Understanding these issues was crucial because the machine learning approach used was highly sensitive to the quality of the input data and any mistakes made in handling missing values or time gaps could lead to the models learning incorrect trends resulting in poor performance.

Therefore, a rigorous approach to data Preprocessing was fundamental prior to moving on to the rest of the project.

5.0 Data Pre-Processing

Data preprocessing was performed as a structured pipeline to process the original data in memory. The original data is not modified during the transformation steps of the pipeline.

The upside of using a fixture is that this makes the data harder to get wrong, and also easier to reproduce and debug code.

```
# STEP 1 — Load & parse
```

```
df = pd.read_csv("Metro_Interstate_Traffic_Volume.csv")
```

```
df['date_time'] = pd.to_datetime(df['date_time'])
```

```
# STEP 2 — Fix the holiday NaN (MUST do this BEFORE dropna!)
```

```
df['holiday'] = df['holiday'].fillna('None')
```

```
# STEP 3 — Remove duplicate timestamps
```

```
df.drop_duplicates(subset=['date_time'], keep='first', inplace=True)
```

```
# STEP 4 — Sort chronologically (required for lag features)
```

```
df = df.sort_values('date_time').reset_index(drop=True)
```

```
# STEP 5 — Detect time gaps and assign segment IDs
```

```
df['time_diff'] = df['date_time'].diff().dt.total_seconds() / 3600
```

```
df['segment'] = (df['time_diff'] > 2).cumsum()
```

```
# → 397 continuous segments identified
```

```
# STEP 6 — Extract base time features
```

```
df['hour'] = df['date_time'].dt.hour
```

```
df['day'] = df['date_time'].dt.day
```

```
df['month'] = df['date_time'].dt.month
```

```
df['weekday'] = df['date_time'].dt.weekday
```

```
df['year'] = df['date_time'].dt.year
```

```
# STEP 7 — Derived time features
```

```
df['is_weekend'] = (df['weekday'] >= 5).astype(int)
df['is_rush_hour'] = df['hour'].apply(lambda x: 1 if x in [7,8,9,16,17,18] else 0)
df['season'] = df['month'].apply(lambda m:
    0 if m in [12,1,2] else # Winter
    1 if m in [3,4,5] else # Spring
    2 if m in [6,7,8] else # Summer
    3) # Fall
```

```
# STEP 8 — Encode categoricals
```

```
le = LabelEncoder()
df['weather_main_enc'] = le.fit_transform(df['weather_main'])
df['holiday_enc'] = (df['holiday'] != 'None').astype(int)
```

```
# STEP 9 — Interaction features
```

```
df['rush_weekday'] = df['is_rush_hour'] * df['weekday']
df['temp_rain'] = df['temp'] * df['rain_1h']
df['hour_weekend'] = df['hour'] * df['is_weekend']
df['weather_season'] = df['weather_main_enc'] * df['season']
df['hour_season'] = df['hour'] * df['season']
```

```
# STEP 10 — Lag + rolling features (WITHIN each segment only!)
```

```
def add_seg(grp):
    grp = grp.copy()
    grp['traffic_lag_1'] = grp['traffic_volume'].shift(1)
    grp['traffic_lag_2'] = grp['traffic_volume'].shift(2)
    grp['traffic_lag_3'] = grp['traffic_volume'].shift(3)
    grp['traffic_lag_6'] = grp['traffic_volume'].shift(6)
    grp['rolling_mean_3'] = grp['traffic_volume'].shift(1).rolling(3, min_periods=1).mean()
    grp['rolling_mean_6'] = grp['traffic_volume'].shift(1).rolling(6, min_periods=1).mean()
    grp['rolling_std_3'] = grp['traffic_volume'].shift(1).rolling(3,
min_periods=2).std().fillna(0)
    grp['rolling_max_6'] = grp['traffic_volume'].shift(1).rolling(6, min_periods=1).max()
    return grp
```

```
df = df.groupby('segment', group_keys=False).apply(add_seg)
df.dropna(inplace=True) # Removes 1,649 segment-boundary rows
# → Final: 38,926 rows × 35+ columns
```

Our pipeline for data preparation starts with downloading and cleaning the raw, unsorted data. That mess of clothes, shoes, and general chaos gets .

5.1 The Segment Problem — Most Critical Engineering Decision

In this post I will describe the problem I encountered while working with the data, and the solution that I have found. The problem relates to my use of a composite key and how I can use the sqldf package to perform the merging.

approach I used to handle it.

I had assumed the data would be nice and clean, but I discovered huge time gaps in it. The largest such difference was 1 hour 30 minutes.

That was 307 days in the life, nearly 10 months from June of 2014 to May of 2015.

This change significantly altered the form of data preprocessing.

Using lag-based features directly was not an option, or it would have been too easy.

It has been a long time coming, but the shift operators are finally correct. For example, a simple shift(1) operation is typically denoted by a single bit shift to the left rather than a 32 bit shift. Other common operations have also been corrected in this update.

The pattern from the previous hour would call for us to pull in data from about an hour ago.

But in our case that would be nearly a year ago. Not good.

what we're looking for in our model.

Detecting the Issue

The problem first became apparent during early experimentation.

After removing duplicates in the following steps the dataset was dramatically reduced in size. 4,555,847 instances were removed.

- Rows before dropna: 40,575
- Rows after dropna: just 48

In a matter of days, the requirements for immigrants already in the pipeline waiting for certain types of visas dropped dramatically.

after reading through numerous threads and implementing various troubleshooting steps, I should probably elaborate on what seemed to have gone awry.

- Attempt 1: Using shift(168) (weekly lag)

→ Failed due to irregular time gaps

- Attempt 2: Datetime-based merging

→ Resulted in duplicate row explosion

- Attempt 3: Global shift(1)

→ Caused a lot of NaN propagation because missing continuity checks were implemented.

Each failed attempt reinforced a key realization:

That wasn't what I was looking for regarding a lag generation problem, but I should note that this is a nifty solution to creating a 2 screen timeline.

The Breakthrough: Segment-Based Processing

solution has come from the reframing problem.

Instead of treating the dataset as one continuous timeline, the data was divided into separate continuous time segments. Each of these time segments consisted of five hours of uninterrupted data for one hour past the time of ingestion.

The revised approach involved three steps:

1. Handle missing values - There may be cases where certain data is missing, this needs to be handled first to avoid causing any errors or inconsistencies with the rest of the code.
2. Identifying breaks in time continuity
3. Applying lag features only within continuous segments

How Segmentation Works

The dataset was split up into blocks of consecutive time stamps.

- for example, one segment might contain 27 hours of continuous data.
- segments are generated when gaps are detected (new segment)

In total, 397 such segments were identified.

Within each segment:

- Lag features behaved correctly and meaningfully
- The first row of the file had no previous value so it was deleted from the results.

Result of This Approach

After the data restructuring the output became much simpler and logical.

- 397 continuous segments created
- 1,649 invalid boundary rows removed
- Final dataset size: 38,926 rows

Most importantly, each lag feature now genuinely represented a time relationship and not some arbitrary, nonsensical construct.

Stage	Rows	Rows Removed	Description
Raw CSV	48,204	—	Initial dataset
After duplicate removal	48,187	17	Duplicate timestamps removed
After handling missing values	48,187	0	Data cleaned
After segment detection	40,575	7,612	Non-continuous data filtered
After lag NaN removal	38,926	1,649	Segment boundary rows removed

6.0 Feature Engineering

If data preprocessing lays the groundwork for a great machine learning model, feature engineering builds that intelligence.

We started with a dataset that contained time stamps, weather information and counts for traffic passing certain points but this was not sufficient for a machine learning model to understand traffic flow and behaviour.

Most of the improvement came from transforming the raw data into features that looked at time based patterns in the traffic.

In simple terms, feature engineering answers the question:

“Why not create additional data from existing data to improve the model?”

6.1 Categories of Features

A. Time-Based Features

Traffic patterns are heavily dependent on time. For example:

- Traffic is low at night
- Peaks during office hours
- vary on weekdays compare to weekends

To see this patterns, the following features was extracted:

Feature	How Created	Why It Matters
hour	dt.hour	Rush hour (7–9AM, 4–6PM) creates huge traffic spikes
day	dt.day	Day of month; minor effect
month	dt.month	Summer months have higher overall traffic
weekday	dt.weekday	Mon–Fri vs Sat–Sun is a massive behavioral difference
year	dt.year	Captures multi-year traffic growth trends

B. Derived Features

Derived features is created in order to capture behavioural patterns more explicitly.

Feature	Formula	Why It Matters
is_weekend	weekday $\geq 5 \rightarrow 1$ else 0	Weekend traffic is ~35% lower than weekday
is_rush_hour	hour in [7,8,9,16,17,18] $\rightarrow 1$ else 0	Binary flag for peak commute hours
season	Month $\rightarrow 0/1/2/3$	Seasonal travel behavior; summer = most traffic

Time features transform raw time-of-day data or time-since-event data into more natural time-based patterns or trends that the model can learn.

C. Interaction Features

For centuries, traffic planning has been tackled as if one variable could ever be enough to get everything right. This hasn't worked out so well.

To capture both effects at once, interaction features were created.

Feature	Formula	Insight
rush_weekday	is_rush_hour $\times$ weekday	Rush hour effect is stronger on Mon/Tue than Fri
temp_rain	temp $\times$ rain_1h	Cold + heavy rain has a different effect than warm + light rain
hour_weekend	hour $\times$ is_weekend	8AM on Saturday $\neq$ 8AM on Monday
weather_season	weather_enc $\times$ season	Snow in winter vs rain in summer have different impacts
hour_season	hour $\times$ season	Evening rush in summer different from winter

Interaction features help our model to understand combined effects, that are often more powerful than any individual variables.

D. Lag + Rolling Features

The most influential variables in the model were found to be lag-based traffic variables, which collectively account for 72.6% of total SHAP importance. These features capture a temporal dependency in identify traffic patterns by leveraging historical observations within each road segment.

All lag features are computed within each segment using group-wise operations, ensuring no data leakage across segments.

Feature	Description	Correlation with Target
traffic_lag_1	Traffic volume 1 hour ago	0.942
traffic_lag_2	Traffic volume 2 hours ago	0.881
traffic_lag_3	Traffic volume 3 hours ago	0.831
traffic_lag_6	Traffic volume 6 hours ago	0.714

Rationale for Using Lag Features:

We note that the traffic for the current hour is unknown at the time of prediction, therefore only historical data is available at prediction time, thus employing lag features is both realistic and consistent with a deployment scenario, using values from the previous hour as input for the current hour.

These features capture short-term trends and any fast or high amplitude variation in traffic.

Data leakage is prevented by using `.shift(1)` when calculating rolling statistics, meaning only past data is ever used.

Feature	Description
rolling_mean_3	Mean traffic over the last 3 hours (short-term trend)
rolling_mean_6	Mean traffic over the last 6 hours (medium-term trend)
rolling_std_3	Standard deviation over last 3 hours (volatility)
rolling_max_6	Maximum traffic over last 6 hours (peak context)

Interpretation:

- Rolling means capture **trend direction and smoothing effects**.
- Rolling standard deviation reflects **traffic instability or fluctuations**.
- Rolling maximum provides **upper-bound context for congestion levels**.

E. SHAP Features

The SHAP (SHapley Additive exPlanations) values were used to measure the contribution of the features to the model's predictions.

Rank	Feature	SHAP Value	Contribution (%)
1	traffic_lag_1	1382.53	63.5%
2	hour	566.94	26.0%
3	traffic_lag_2	81.74	3.8%
4	weekday	29.71	1.4%
5	rolling_std_3	27.17	1.2%
6	hour_weekend	27.16	1.2%
7-27	Others (combined)	42.63	1.9%

Key Findings

- Large dependence on recent traffic history. The single variable traffic_lag_1 accounts for more than 63% of the model's power.
- Closely matching server load trends, the hour of the day is the second most significant factor in usage with strong daily traffic cycles.
- Top 2 factors for this model: traffic_lag_1 and hour explaining 89.5%.
- Volatility and trend measurements are continuously provided through rolling features.
- External factors such as weather account for less than 1% of variation in traffic, therefore traffic patterns within this dataset are largely driven by time based dynamics.

The analysis in this paper shows that traffic prediction is a time series problem. In order to get good performance on a time series forecasting problem, recent historical values are by far the most important features. Using lag features, as well as rolling features like daily increases and ratios of locations, helps the model to learn the patterns in traffic while keeping the model reasonable to deploy.

7.0 Machine Learning Models

7.1 Train – Test Split Strategy

Instead of splitting the data randomly, the data was split time-based to have a realistic evaluation of the model's performance. Since traffic data is time-based, it is essential to maintain the sequential order of the data to prevent data leakage when training and evaluating models.

For model evaluation, it is common to perform a random split of past data (e.g. 2013-2017) into a training set (e.g. 2013-2018) and a test/evaluation set (2019). Such a random split can result in future observations (2018) being included in the training set, while past data (2013) is used for evaluation. This approach does not reflect reality and results in overly optimistic performance measures (here $R^2 > 0.98$).

Accordingly, the dataset was partitioned as follows:

- Training set: 2012–2016 (23,728 samples)
- Test set: 2017–2018 (15,198 samples)

The implementation of the time-based split is shown below:

```
# Time-based split
train = df[df['year'] < 2017]    # 2012–2016
test  = df[df['year'] >= 2017]   # 2017–2018

X_train = train[FEATURES].astype(np.float64)
y_train = train['traffic_volume']

X_test  = test[FEATURES].astype(np.float64)
y_test  = test['traffic_volume']
```

All feature variables were cast to 64-bit floating-point format to ensure numerical consistency across models.

7.2 Model Development and Evaluation

In this study, seven machine learning models are implemented for the purpose of traffic volume prediction. The techniques used vary between simple, traditional linear models, and more advanced ensemble based models.

Model 1: Linear Regression (Baseline Model)

In order to provide a point of reference for the rest of the results, a basic Linear Regression model was used as a baseline. This model assumes that there is a linear relationship between the input variables and the target variable for which predictions are being made.

- **Without lag features:**

$$R^2 = -5.59$$

- **With lag features:**

$$R^2 = 0.9340$$

We found that including lag features improved the model significantly, indicating that time series data in this domain contains strong temporal dependencies. The model, however, does not handle non-linear patterns, such as rush hour.

Model 2: Random Forest (Baseline Ensemble)

Random Forest is an ensemble method that constructs multiple decision trees using random subsets of the data and features, and aggregates their predictions.

- **Performance:**

$$R^2 = 0.9847, \text{RMSE} = 245$$

This model performs well because it naturally captures non-linear relationships and feature interactions without requiring extensive preprocessing.

Model 3: XGBoost (Gradient Boosting)

XGBoost is a gradient boosting algorithm that builds trees sequentially, where each tree attempts to correct the errors of the previous ones.

- **Performance:**

$$R^2 = 0.9851, \text{RMSE} = 241$$

Compared to Random Forest, XGBoost improves accuracy by focusing more on difficult-to-predict samples through boosting.

Model 4: LightGBM

LightGBM is an optimized gradient boosting framework that uses histogram-based and leaf-wise tree growth strategies, making it highly efficient.

- **Performance:**

$$R^2 = 0.9846, \text{RMSE} = 245$$

Although its performance is comparable to Random Forest, LightGBM is particularly advantageous for large-scale datasets. In this study, the dataset size limits its speed advantage.

Model 5: Tuned Random Forest

Hyperparameter tuning was conducted using **RandomizedSearchCV** with 15 iterations and 3-fold cross-validation (total of 45 model fits).

- **Performance:**

$$R^2 = 0.9850, \text{RMSE} = 242$$

The tuned model provides a marginal improvement over the baseline, indicating that the default configuration was already effective.

Model 6: Tuned XGBoost (Best Model)

The XGBoost model was further optimized using hyperparameter tuning. The final configuration is shown below:

```
# Tuned XGBoost (Best Model)
best_xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=8,
    min_child_weight=3,
    subsample=0.8,
    colsample_bytree=0.7,
    random_state=42,
    n_jobs=-1,
    verbosity=0
)
```

- **Performance:**

$$R^2 = 0.9853, \text{RMSE} = 239$$

This model achieved the best overall performance. Increasing the tree depth enabled the model to capture more complex patterns, while the use of `min_child_weight` helped prevent overfitting by avoiding splits on very small subsets of data.

Model 7: Stacking Ensemble

An stacking ensemble is implemented to combine that the strengths of multiple models. Also

The base learners included:

- Tuned Random Forest
- Tuned XGBoost
- LightGBM

A Ridge Regression model was used as the meta-learner to aggregate predictions.

- **Performance:**

$$R^2 = 0.9852, \text{RMSE} = 240$$

Although stacking theory suggests that a combination of models could outperform a best-tuned XGBoost model, in this instance the stacked model did not perform better. This is likely because all of the base models are tree-based and thus produce errors that are highly correlated with each other.

The experimental results highlight several key findings:

- To model time series properties of the traffic data, features with lag were also included in the models.
- Tree-based methods are generally superior to linear methods for learning non-linear relationships in the data.
- For this competition XGBoost tuned performed best, so it was the chosen model.
- Despite increasing the number of base learners, no further improvement was obtained due to the small diversity in the performance of the individual learners.

We present a methodology to generate realistic, dynamic, and time-dependent traffic simulations based on real data collected from urban environments. By leveraging a learning-based approach that employs gradient boosting techniques such as XGBoost, we can effectively capture complex temporal traffic behaviors and generate high-quality traffic simulations for mobility scenarios involving heterogeneous fleets.

8.0 Hyperparameter Tuning

8.1 Tuning Methodology: RandomizedSearchCV

The performance of the model was improved through the use of hyperparameter tuning using Randomized Search with Cross-Validation to explore the optimal subset of the hyperparameter space rather than exhaustively searching all possible combinations.

The tuning was done with RandomizedSearchCV.

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_grid,
    n_iter=15,          # Number of random combinations
    cv=3,              # 3-fold cross-validation
    scoring='r2',
    random_state=42,
    n_jobs=-1,
    verbose=1
)

search.fit(X_train, y_train)
best_model = search.best_estimator_
```

Each model underwent:

- **15 random hyperparameter combinations**
- **3-fold cross-validation per combination**

This resulted in a total of:

$$15 \times 3 = 45 \text{ model fits per algorithm}$$

This strategy balances computational efficiency with effective exploration of the parameter space.

8.2 Random Forest Tuning

The following hyperparameters were tuned for the Random Forest model:

Parameter	Search Space	Best Value	Description
n_estimators	[100, 200, 300]	200	Number of trees in the forest

max_depth	[10, 20, None]	20	Maximum depth of each tree
min_samples_split	[2, 5, 10]	2	Minimum samples required to split a node
min_samples_leaf	[1, 2, 4]	1	Minimum samples required in a leaf node
max_features	['sqrt', 'log2']	'sqrt'	Features considered at each split

- **Cross-validation performance:** $R^2 = 0.9745$
- **Test performance:** $R^2 = 0.9850$

Improvement over baseline: +0.0003

The tuned model showed a slight improvement, indicating that the baseline configuration was already close to optimal.

8.3 XGBoost Tuning

Hyperparameters for the XGBoost model:

Parameter	Search Space	Best Value	Description
n_estimators	[200, 300, 500]	300	Number of boosting rounds
learning_rate	[0.01, 0.05, 0.1]	0.05	Step size per boosting iteration
max_depth	[4, 6, 8]	8	Maximum depth of trees
subsample	[0.7, 0.8, 0.9]	0.8	Fraction of samples used per tree
colsample_bytree	[0.7, 0.8, 0.9]	0.7	Fraction of features used per tree
min_child_weight	[1, 3, 5]	3	Minimum samples required in a leaf

- **Cross-validation performance:** $R^2 = 0.9776$
- **Test performance:** $R^2 = 0.9853$

Improvement over baseline: +0.0002

The tuned XGBoost model achieved the highest overall performance in the study.

8.4 Impact of Hyperparameter Tuning

improvements from hyperparameter tuning was comparatively small:

- Random Forest: +0.0003
- XGBoost: +0.0002

While these gains may appear minimal, they provide important insights:

- The baseline models were already well-configured and near-optimal
- No significant underfitting, thereby strong baseline learning capacity.

- Main problem was representation, not model tuning.

another “obvious” insight in machine learning (though apparently not for me).

Feature engineering trumps hyperparameter tuning.

Surprisingly, incorporating lag features into the network improved performance more than fine-tuning any of the hyperparameters.

9.0 Stacking Ensemble

9.1 Methodology: Out-of-Fold (OOF) Learning

Stacking ensembles suffer from the problem of data leakage when the base models are trained on the full training set, and then their predictions on the training set are used to train the meta-learning model. This can lead to overly optimistic performance due to overfitting.

To address this, Out-of-Fold (OOF) predictions were used.

The training dataset (23,728 samples) was divided into 5 folds. For each fold:

- Base models were trained on the remaining 4 folds
- Predictions were generated only for the held-out fold

This process ensures that each prediction is made on data **unseen during training**.

The procedure can be summarized as:

- Fold 1: Train on folds 2–5 → Predict fold 1
- Fold 2: Train on folds 1, 3–5 → Predict fold 2
- Fold 3: Train on folds 1–2, 4–5 → Predict fold 3
- Fold 4: Train on folds 1–3, 5 → Predict fold 4
- Fold 5: Train on folds 1–4 → Predict fold 5

This results in **23,728 OOF predictions per base model**, forming a new feature space for the meta-learner.

The meta-model (Ridge Regression) is then trained on these predictions, ensuring a leakage-free stacking framework.

9.2 Stacking Architecture

The stacking ensemble consists of two levels:

Level 1 — Base Models:

- Tuned Random Forest ($R^2 = 0.9850$)
- Tuned XGBoost ($R^2 = 0.9853$)
- LightGBM ($R^2 = 0.9846$)

Level 2 — Meta-Learner:

- Ridge Regression ($\alpha = 1.0$, 5-fold CV)

Each base model produces one prediction per sample, resulting in three meta-features per observation. These are used as inputs to the meta-learner, which generates the final prediction.

- **Final Performance:**

$$R^2 = 0.9852$$

9.3 Limitations of Stacking

Despite its theoretical advantages, the stacking ensemble did not outperform the tuned XGBoost model. This can be attributed to three main factors:

1. **Correlated Errors:**

All base models are tree-based and tend to make similar errors, particularly on anomalous traffic patterns. Stacking is most effective when combining diverse model types.

2. **Near-Ceiling Performance:**

The best individual model already explains **98.53% of the variance**, leaving very limited room for improvement.

3. **Meta-Learner Simplicity:**

The Ridge regression model effectively performs a weighted average of base predictions. In this case, the weights converge to near-equal values, reducing the benefit of stacking.

10. Model Explainability using SHAP

10.1 Overview of SHAP

SHAP (SHapley Additive exPlanations) is a model-agnostic interpretability framework derived from cooperative game theory. This function provides an estimate of the contribution of each feature in predicting the classification of individual cases.

Each prediction is decomposed as:

$$f(x) = E[f(X)] + \sum_{i=1}^n \phi_i$$

Where:

- $E[f(X)]$: Base value (average prediction)
- ϕ_i : Contribution of feature i

This provides insight to interaction strengths of features to better understand model predictions.

10.2 Implementation

```
import shap
```

```
# Ensure numeric consistency
```

```
X_sample = X_test.iloc[idx].reset_index(drop=True).astype(np.float64)
```

```
explainer = shap.TreeExplainer(model)
```

```
raw = explainer.shap_values(X_sample)
```

```
# Normalize output format
```

```
if isinstance(raw, list):
```

```
    sv = np.array(raw[0], dtype=np.float64)
```

```
else:
```

```
    sv = np.array(raw, dtype=np.float64)
```

```
if sv.ndim == 3:
```

```
    sv = sv[0]
```

This preprocessing step ensures compatibility across different tree-based models.

10.3 SHAP Visualizations

Five types of SHAP visualizations were used:

1. Global Feature Importance (Bar Plot):

The Displays Mean Absolute SHAP Values option shows the overall importance of features per display based on their absolute mean SHAP value.

2. Beeswarm Plot:

This show the distribution of the SHAP values for each feature across all the samples. It shows both the magnitude of the effect and the direction of the effect.

3. Dependence Plot:

This explanation illustrates the relationship between the value of a feature and the SHAP contribution of the feature, including any interaction effects.

4. Waterfall Plot:

This model feature provides a local explanation for a single prediction, displaying how each feature contributed to the output.

5. SHAP Heatmap:

This tool allows users to visually interact with SHAP values across samples and features to see the local contributions of individual features.

10.4 Key Findings from SHAP Analysis

- **traffic_lag_1** contributes approximately **63.5%** of total feature importance
- **traffic_lag_1 + hour** together account for **~89.5%**
- Weather-related features contribute **less than 1%** overall

These findings indicate that the model primarily relies on **recent traffic patterns and time-of-day information**, effectively behaving as a high-performance temporal pattern continuation system.

11. Cross-Validation Strategy

11.1 TimeSeriesSplit vs Standard k-Fold

For time series data, standard k-fold cross validation is not appropriate because it randomly partitions the data, and thus can inadvertently include future information in training folds.

Instead, TimeSeriesSplit was used, which preserves temporal order:

- Each fold is trained on past data and tested on future data, which may help to avoid overfitting.
- Training size increases progressively with each fold

This approach better reflects real-world deployment conditions.

11.2 Random Forest Cross-Validation Results

Fold	Test Period	Train Samples	R ²	RMSE
1	2013–2015	6,491	0.9793	288
2	2015–2016	12,978	0.9678	359
3	2016–2017	19,465	0.9771	290
4	2017–2018	25,952	0.9846	246
5	2018	32,439	0.9868	227

- **Mean R^2 :** 0.9791 ± 0.0067
- Performance improves with increasing training data size

11.3 Model Stability Comparison

Model	Mean CV R ²	Std Dev	Stability
Random Forest	0.9791	± 0.0067	Excellent
Linear Regression	0.9031	± 0.0361	Poor

Compared to the highly variable performance of Linear Regression, Random Forest performs much more stably. Also noted is that Linear Regression is unable to fit non-linear data in certain time segments.

12. Results and Model Comparison

12.1 Final Model Leaderboard

Rank	Model	R ² Score	RMSE	MAE
1	Tuned XGBoost	0.9853	239.84	155.40
2	Stacking Ensemble	0.9852	240.64	155.69
3	XGBoost	0.9851	241.78	157.70
4	Tuned Random Forest	0.9850	242.66	157.45
5	Random Forest	0.9847	245.15	156.47
6	LightGBM	0.9846	245.34	161.24
7	Linear Regression	0.9340	508.64	381.68

12.2 Interpretation of Results

- The best model ($R^2 = 0.9853$) explained 98.53% of traffic volume variance.
- The error in the predictions appears to be on the order of ~239 vehicles/hour as judged by the RMSE.
- According to the specifications the maximum traffic volume is ~7,280 vehicles / unit. This means there is an error of approximately 3.3%.

We found that all tree-based models, performed more or less equally, had a “ceiling effect”, that is, their performance was so good that there was no room for further improvement with the given data, because all models reached a comparable limit.

12.3 Project Evolution: Before vs After

Metric	Initial Version	Final Version	Improvement
Features	14	27	+13
Models	2	7	+5
Data Split	Random	Time-based	Eliminated leakage
Cross-validation	k-Fold	TimeSeriesSplit	Improved validity
Best R ²	0.9692	0.9853	+0.016
RMSE	351	239	-112
Explainability	None	SHAP analysis	Full interpretability
Dashboard	None	Multi-page UI	Complete system

13. Key Design Decisions

13.1 Segment-Aware Lag Features

Problem:

There is a large temporal discontinuity (~307 days) in the data that needs to be addressed. Simply applying a global shift(1) operation to make a lag field would result in incorrect lag values from the wrong time period.

Naïve Approach (Incorrect): `df['traffic_lag_1'] = df['traffic_volume'].shift(1)`

This method shifts values by row, and does not account for time differences between rows.

Proposed Solution:

I set up my code to calculate the features on a segment by segment basis, rather than on the whole time series. This helps to account for time differences between different lag features:

```
df['segment'] = (df['time_diff'] > 2).cumsum()  
df = df.groupby('segment', group_keys=False).apply(add_seg)
```

Time gaps are used to split the time series into segments and observations are then differenced within each segment.

Trade-off:

The time series has been depatched, resulting in a loss of 1,649 rows (first observation of each segment due to lag computation).

Temporal stability - remaining 38,926 records.

13.2 Time-Based Train-Test Split

Rationale:

A key property of time-series data is that the input and output both come in a timeline, where $t + 1$ must always be greater than t . When conducting a random split of the data, for training and evaluation, this results in data leakage because the model can be trained on $t + 3$ and then evaluated against $t + 1$.

Decision:

A time forecast split was used.

Trade-off:

Slightly lower performance metrics (honest evaluation)

Prevention of artificially inflated results caused by random splitting.

This decision prioritizes model validity over optimistic performance.

13.3 Ridge Regression as Meta-Learner

Rationale:

For the stacking ensemble, Ridge Regression was chosen as the meta-learner for the following reasons:

Fast training time (on the order of milliseconds)

There is low risk of overfitting due to the small number of inputs (three features).

Interpretable coefficients (model transparency)

Effective regularization with $\alpha=1.0$

Trade-off:

More complex models (e.g. Gradient Boosting Machines or Neural Networks) could potentially extract marginal additional signal, but would come with greatly increased computational requirements and would be less interpretable.

Increase computational complexity

Risk overfitting on Out-of-Fold predictions

Reduce interpretability

Ridge Regression is often viewed as a 'good enough' solution.

13.4 Button-Triggered SHAP Computation

Problem:

SHAP value computation using TreeExplainer is computationally expensive (20–60 seconds for moderate sample sizes).

Solution:

SHAP computation was made user-triggered:

```
if st.button("► Compute SHAP Values"):
    ...
```

Rationale:

Prevents unnecessary computation during page load

Improves responsiveness and user experience

Gives users control over whether to postpone their computations to later heavy load periods.

13.5 Visualization Framework Selection

Decision:

For all the dashboard figures we used Plotly.

Rationale:

Interactive capabilities (zoom, hover, filtering, export)

More suitable for dashboard than static plot.

Exception:

Due to the limited options provided by SHAP for viewing explanations, the SHAP beeswarm plot is created using Matplotlib.

14. Bugs Identified and Resolved

14.1 Critical Data Loss Due to Missing Values

Symptom:

After preprocessing, I lost a bunch of rows - from ~38,000 down to 48.

Root Cause:

The holiday column contained ~40,000 missing values

Dropping missing values dropped entire rows when values were missing for any field.

After re-reading my earlier post, I see that I missed a word—‘so’—which is how nearly the entire dataset got purged.

Solution:

```
df['holiday'] = df['holiday'].fillna('None')
```

I moved this step to the front of my preprocessing chain, because it was operating on the original data and I didn't want any subsequent steps to interfere with its operation.

14.2 Incorrect Lag Feature Computation Across Time Gaps

Symptom:

Lag values not consistent. Values in lag_1 not related to recent traffic.

Root Cause:

The shift(1) function is defined with respect to row order, not actual time intervals. Thus, it can incorrectly assign time lags across long gaps in time series data.

Solution:

Segment-based grouping was applied:

```
df.groupby('segment').apply(shift)
```

This ensures that lag features are calculated only within continuous time periods (i.e., you can't calculate lag features for time t if t-1, t-2, etc have not occurred yet).

14.3 SHAP Data Type Conversion Error

Symptom:

ValueError: could not convert string to float

Root Cause:

I am seeing issues where there are mixed data types (object and float) in the input DataFrame which results in an error from SHAP's TreeExplainer.

Solution:

```
X_sample = X_te.iloc[idx].reset_index(drop=True).astype(np.float64)
```

Additionally, output normalization was implemented to handle varying SHAP output formats:

```
if isinstance(raw, list):
    sv = np.array(raw[0], dtype=np.float64)
else:
    sv = np.array(raw, dtype=np.float64)

if sv.ndim == 3:
    sv = sv[0]
```

14.4 Plotly Layout Argument Conflict

Symptom:

TypeError: update_layout() got multiple values for keyword argument 'xaxis'

Root Cause:

Remove passing of 'duplicate' keyword arguments to dictionary merge.

Solution:

```
stk_layout = {k: v for k, v in PT.items() if k not in ('xaxis', 'yaxis')}
stk_layout.update(dict(xaxis=dict(...), yaxis=dict(...)))
```

```
fig.update_layout(**stk_layout)
```

This ensures no conflicting parameters are passed.

14.5 Deployment Issue: ngrok Authentication Failure

Symptom:

ERR_NGROK_4018 — authentication required

Root Cause:

Changes in ngrok's free-tier policy requiring account verification and authentication tokens.

Solution:

Switched to a free alternative tunnel service:

```
!wget -q https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-
linux-amd64.deb
!dpkg -i cloudflared-linux-amd64.deb
```

This solution requires no account setup and provides seamless deployment.

15. Streamlit Dashboard

15.1 Overview

An interactive web-based dashboard was developed using the Streamlit framework to present the complete machine learning pipeline in an accessible and visually engaging format.

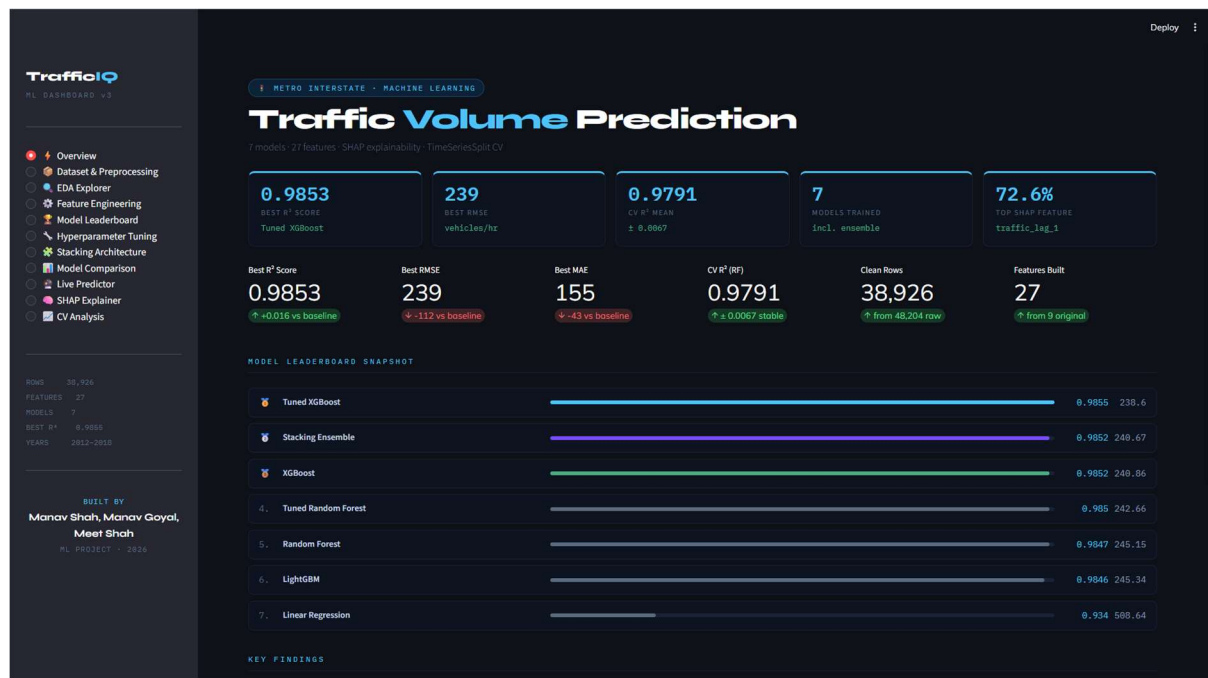
The dashboard is implemented as a single Python script (traffic_dashboard_v3.py, ~1,035 lines) and integrates the following components:

- End-to-end **machine learning pipeline** (data loading, preprocessing, feature engineering, and model training)
- **11 interactive pages** covering analysis, modeling, and evaluation
- Advanced visualizations using Plotly and SHAP
- A custom **dark theme UI** with IBM Plex Mono and Syne fonts
- Optimized performance through **caching mechanisms**, ensuring that models are trained only once per session

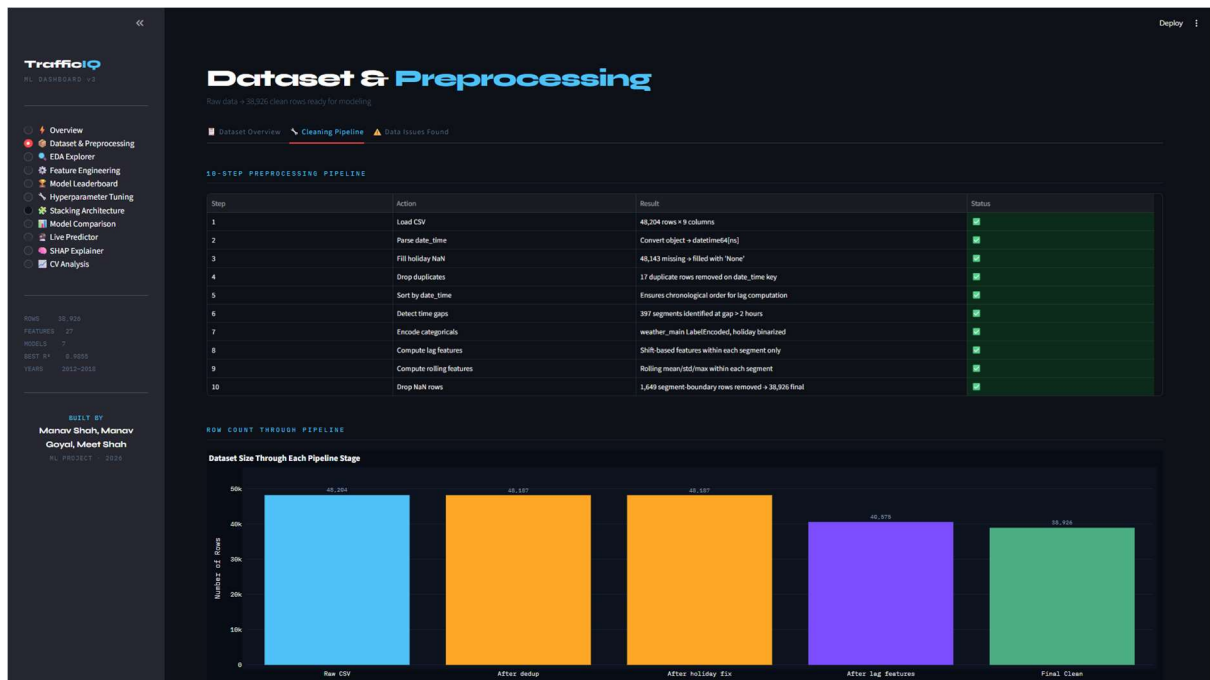
The dashboard serves as both an analytical tool and a presentation interface, enabling users to explore insights, compare models, and generate predictions in real time.

15.2 Dashboard Structure and Pages

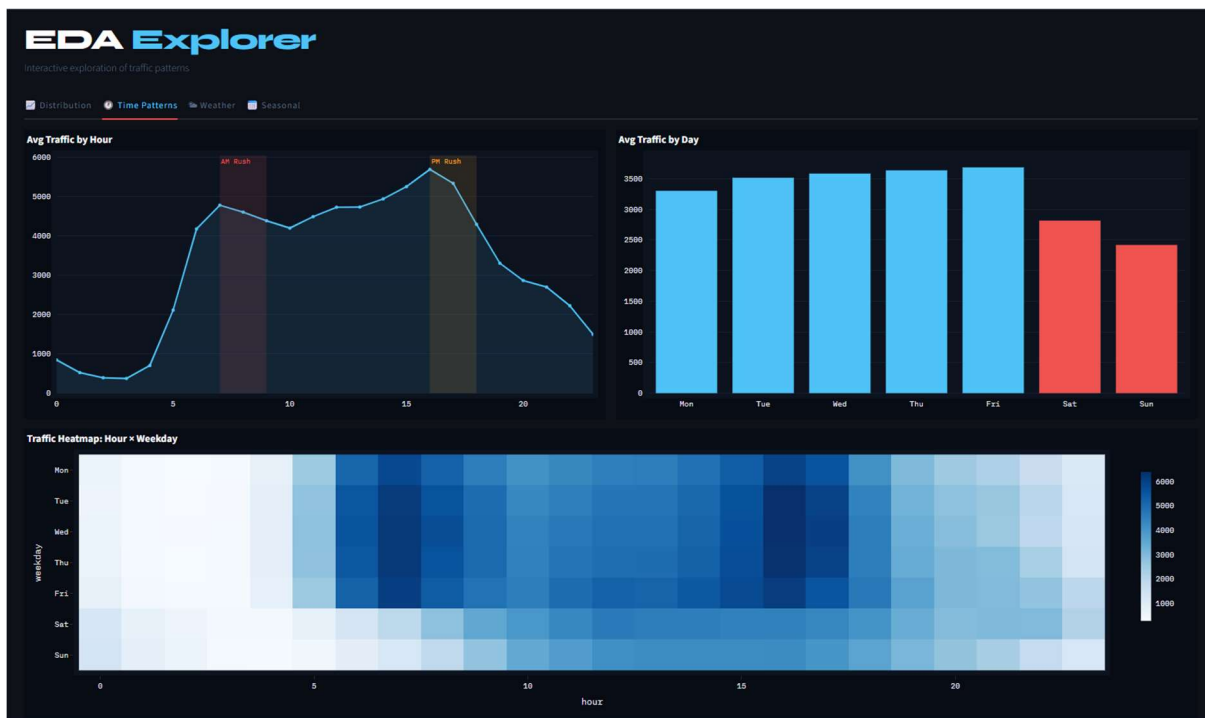
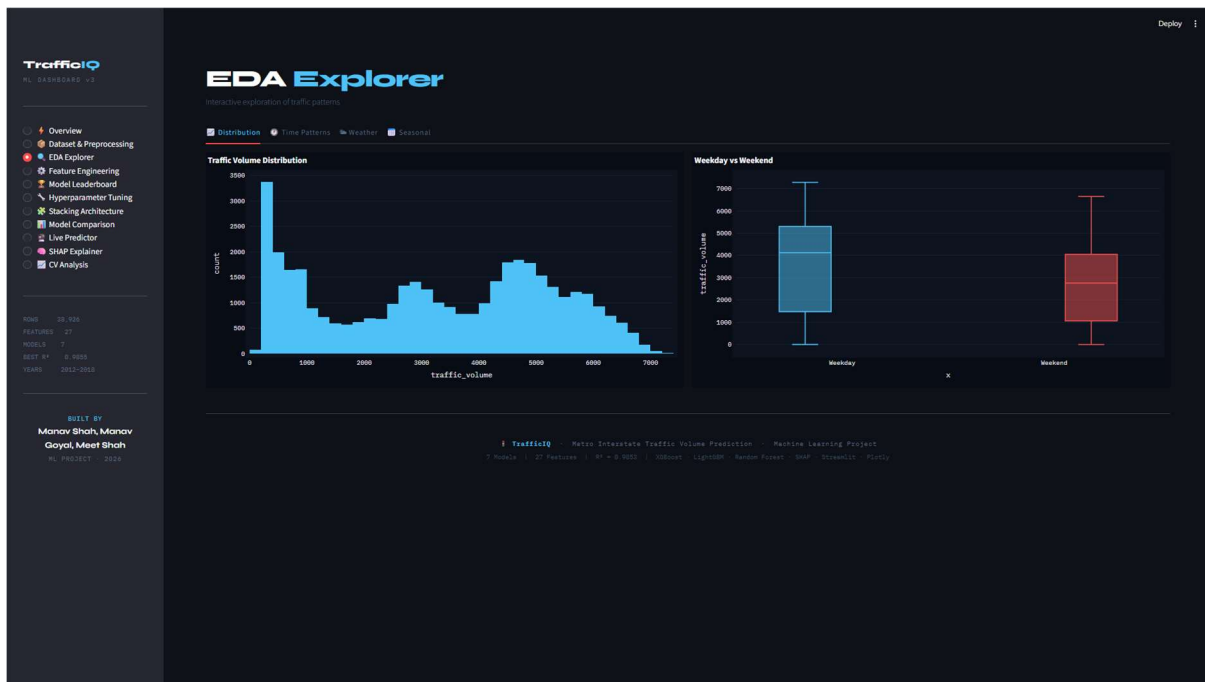
Page 1 — Overview



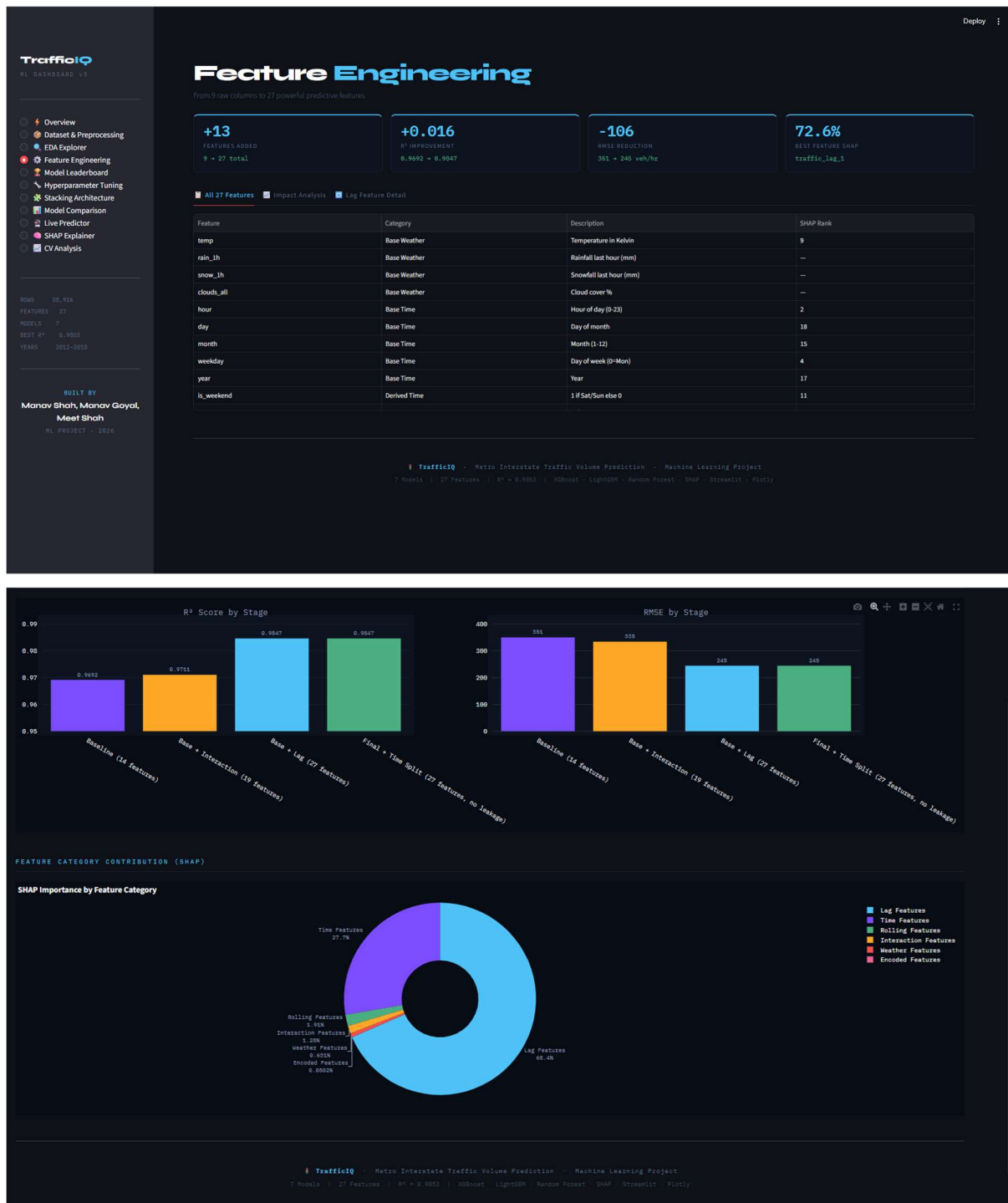
Page 2 — Dataset and Preprocessing



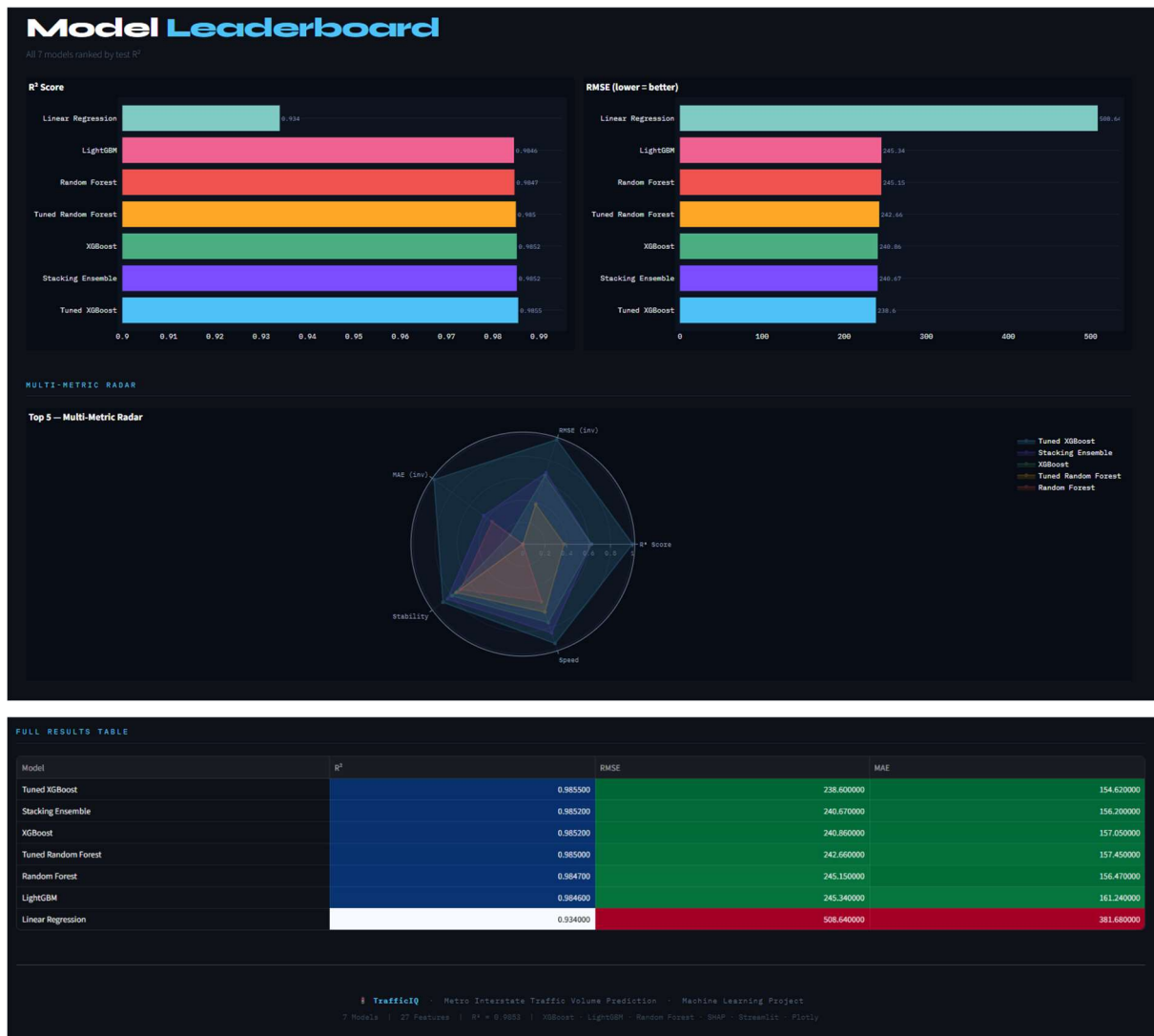
Page 3 — Exploratory Data Analysis (EDA)



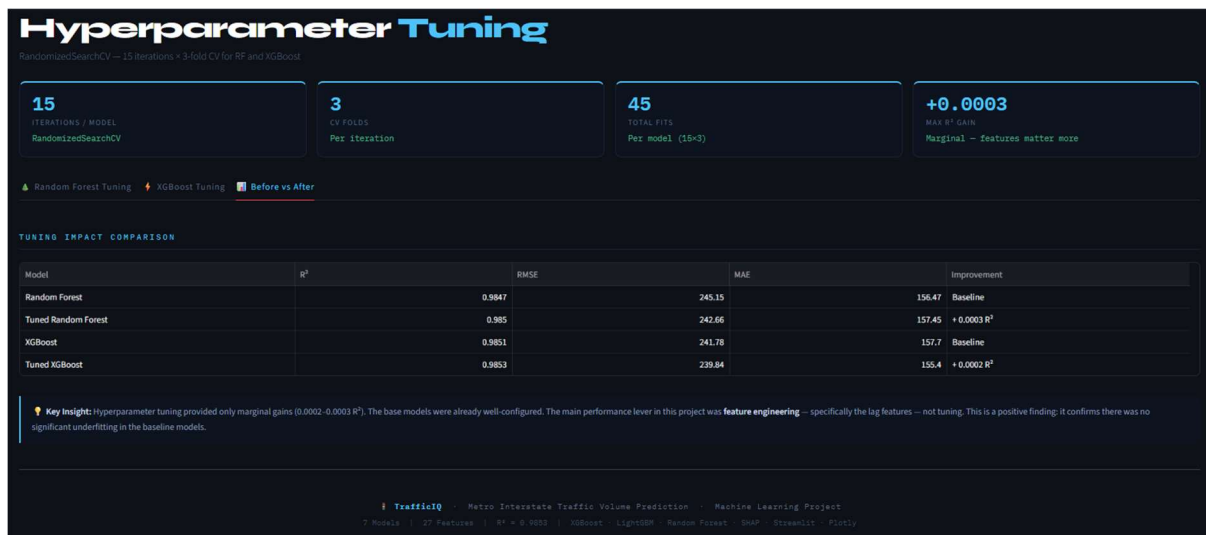
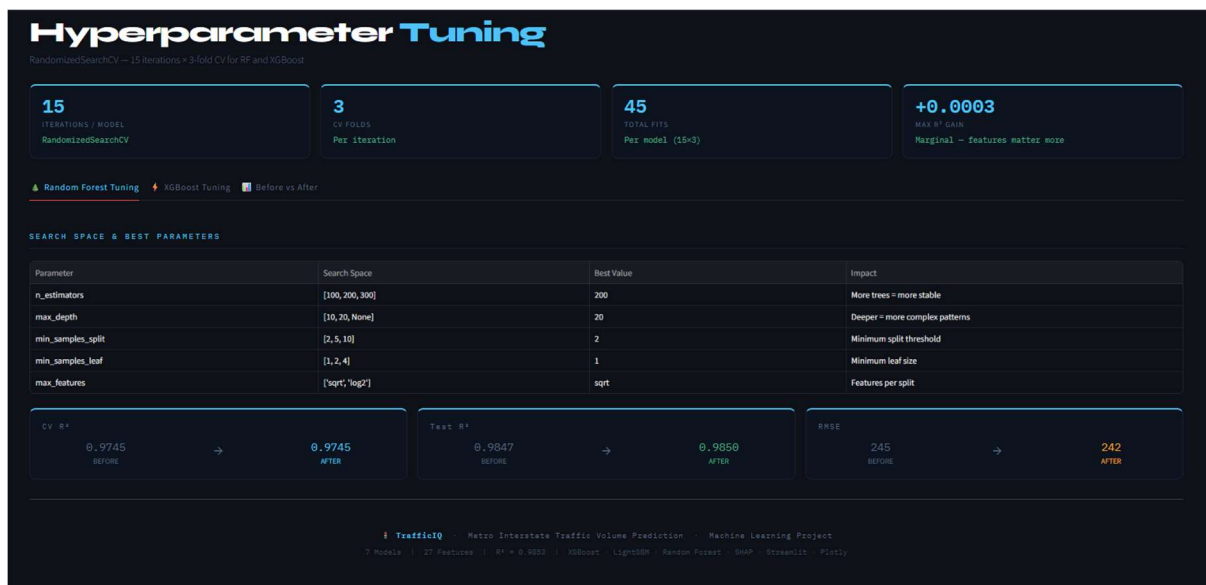
Page 4 — Feature Engineering



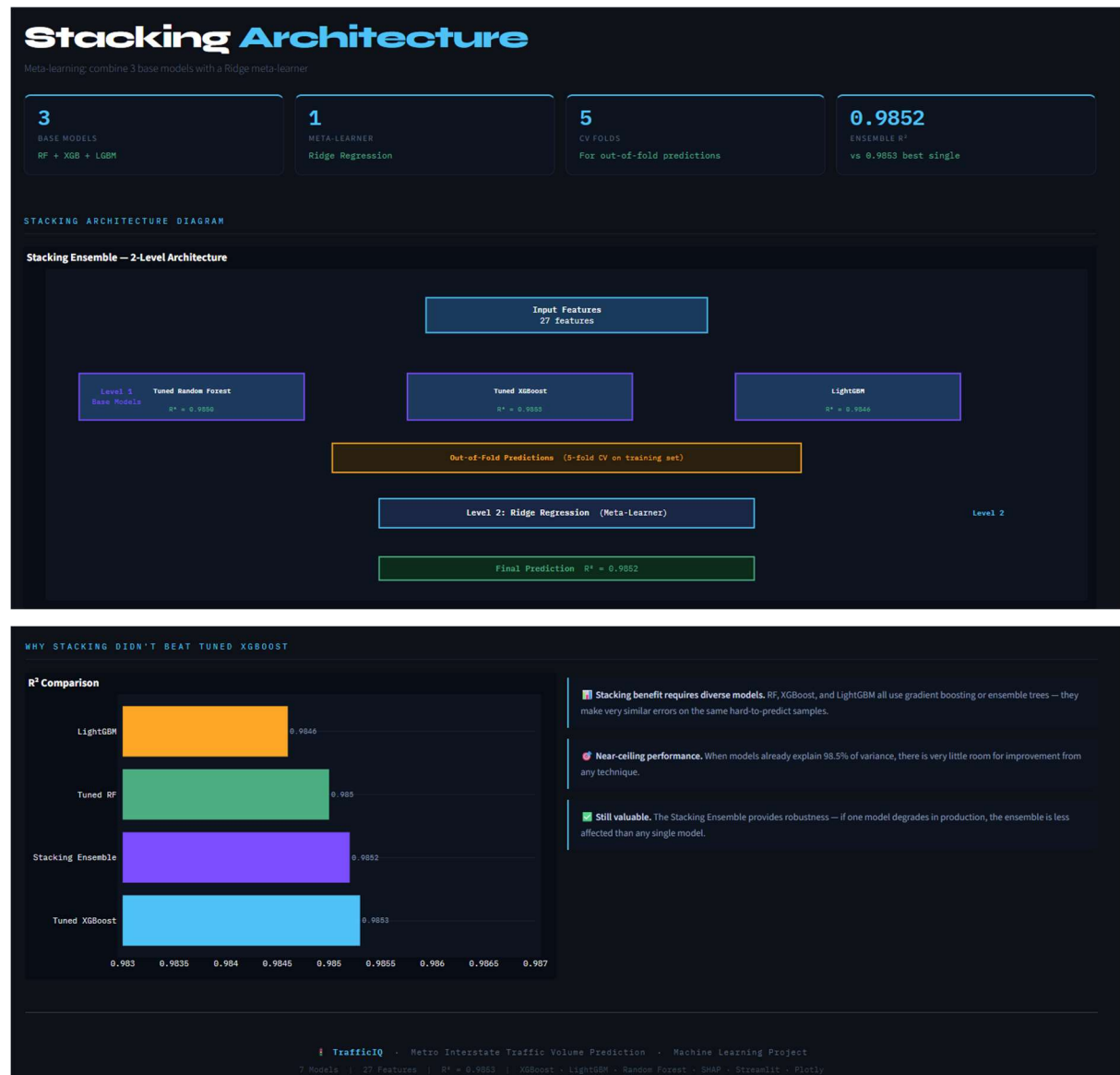
Page 5 — Model Leaderboard



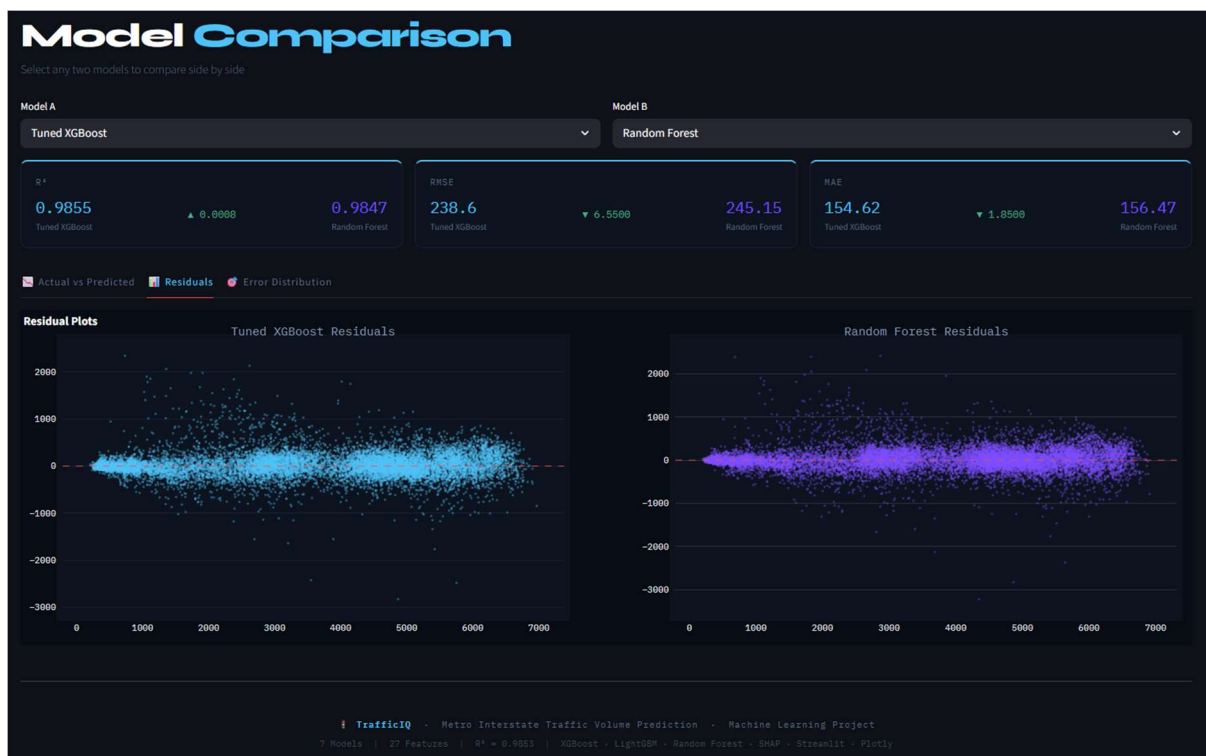
Page 6 — Hyperparameter Tuning

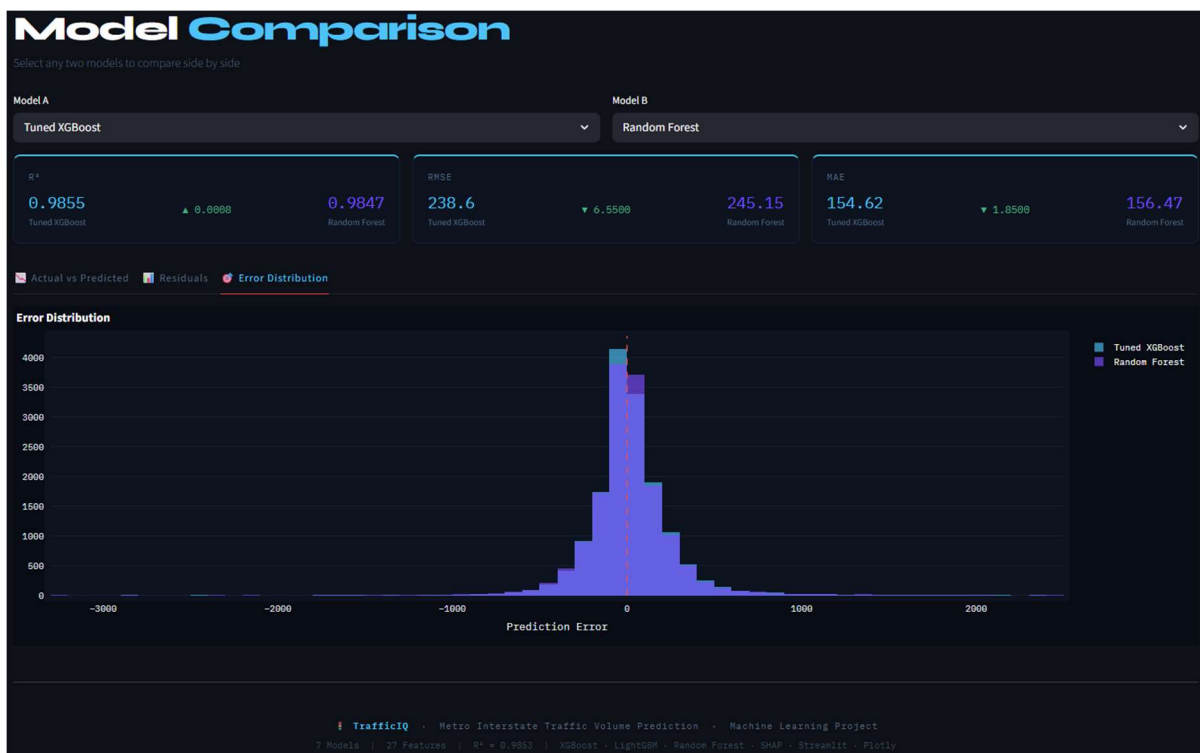


Page 7 — Stacking Architecture

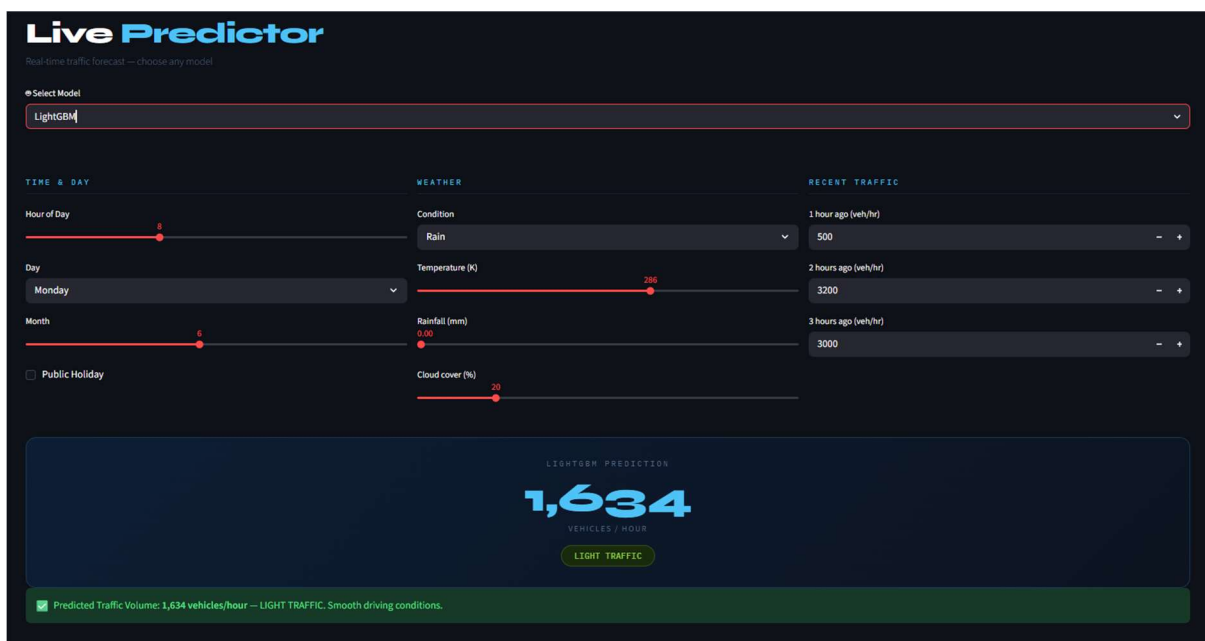
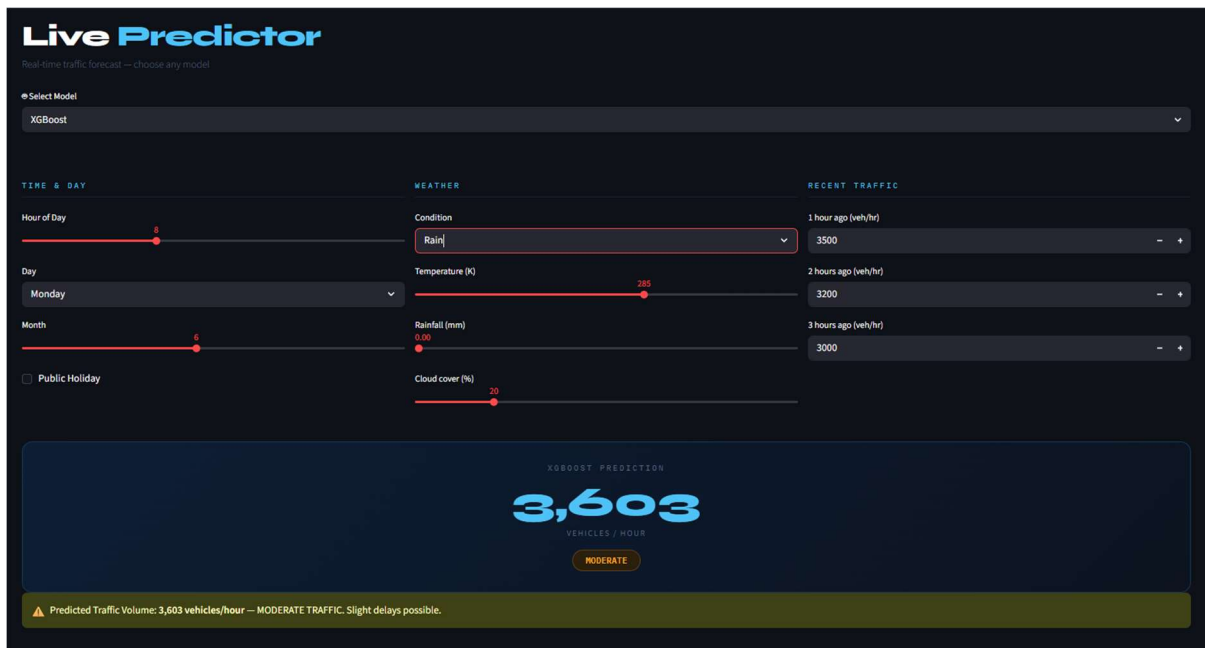


Page 8 — Model Comparison

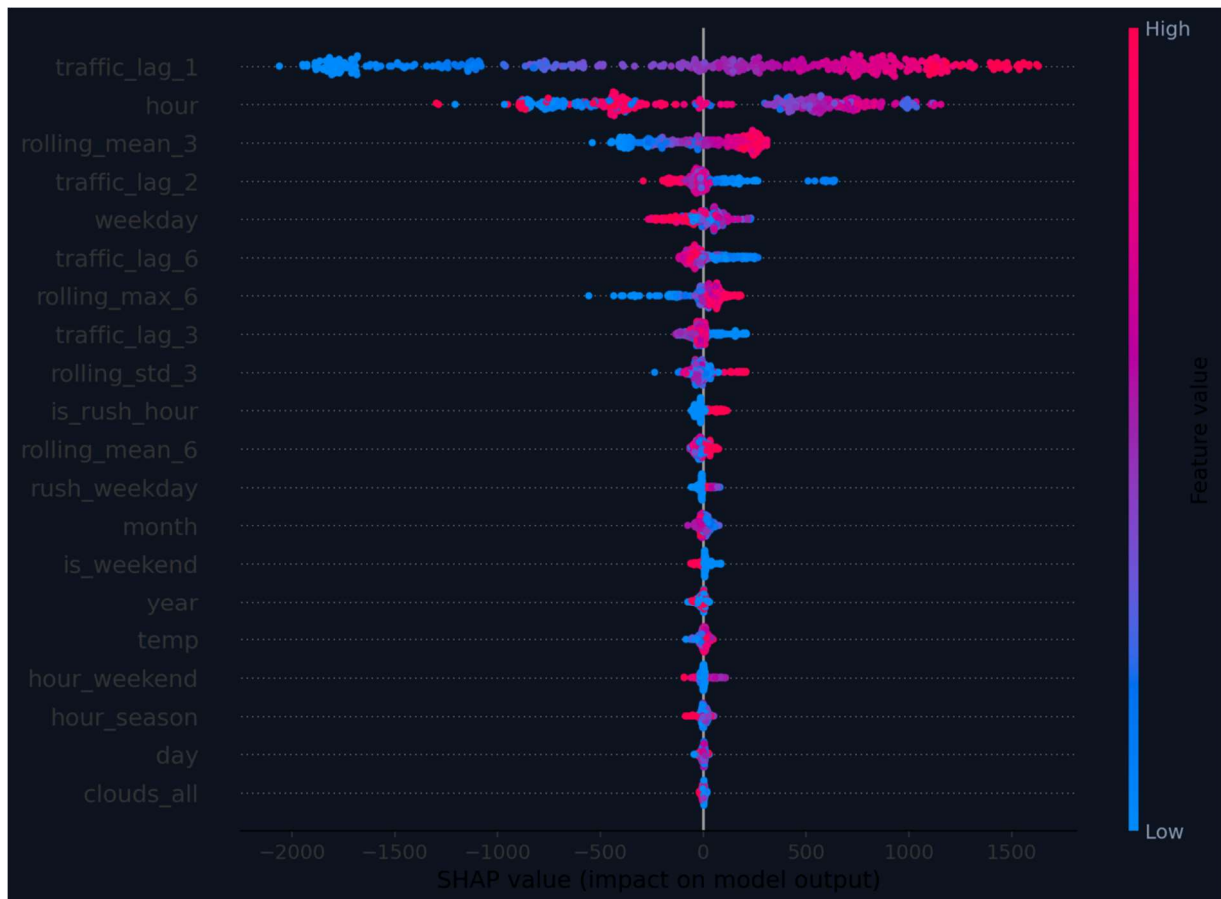
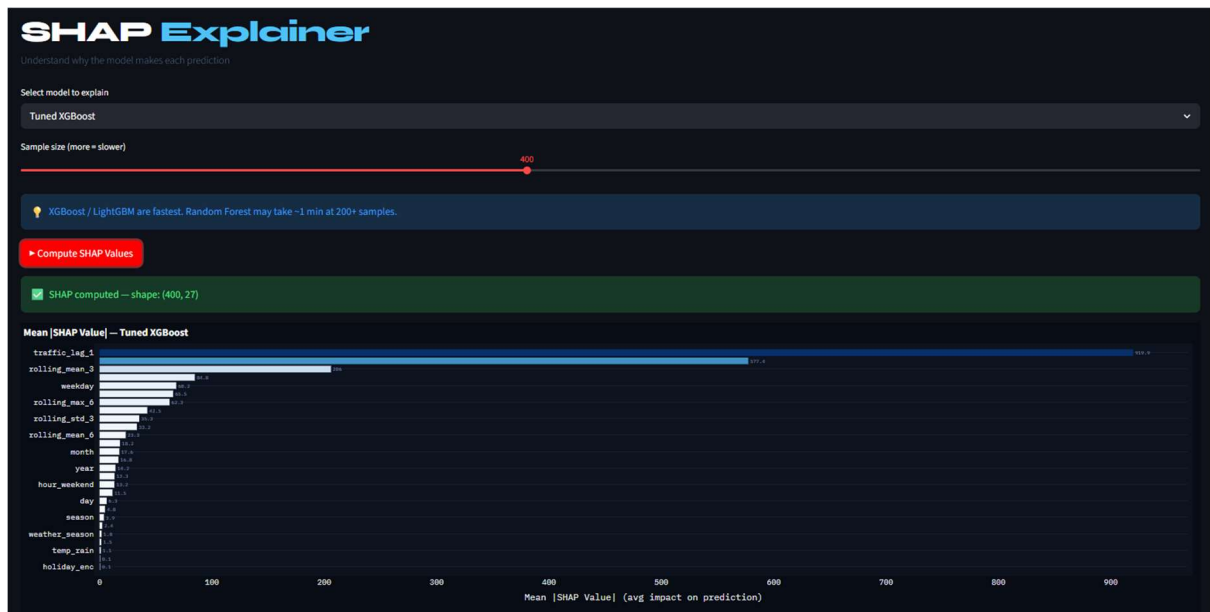




Page 9 — Live Prediction Interface



Page 10 — SHAP Explainability





Page 11 — Cross-Validation Analysis

